Métricas DORA em projetos open-source: o que se pode medir com dados públicos do GitHub

João Santana Lopes[1], **Pedro Maia**[1], **Arthur Candian**[1]

[1]Pontifícia Universidade Católica de Minas Gerais (PUC Minas)
Laboratório de Experimentação de Software

joaovslopes20@gmail.com, pedromaiasilva17@gmail.com,
acandian15@gmail.com

Abstract. *The DORA metrics describe software delivery performance through four indicators, but they were established from self-reported surveys of corporate teams, which makes them neither auditable nor obviously applicable to open-source projects. We compute the four metrics for 350 popular GitHub repositories over a fixed 12-month window, using only public API data, with releases as a proxy for deployments and failed CI runs as a proxy for failed changes. We validate the proxies against a gold sample of 60 repositories and 299 releases labelled independently by three raters. The median deployment frequency is 0.42 releases per week and the median recovery time is 2.56 hours, but the change failure rate depends critically on how failure is operationalised: the CI proxy and the delivery proxy correlate at only* $\rho = 0.107$ *and disagree on the DORA category of 20.6% of the sample. A sensitivity analysis over three definitions of deployment moves between 25% and 33% of repositories across categories. We conclude that DORA classifications derived from public repository data are contingent on operational choices that are rarely made explicit.*

Resumo. *As métricas DORA descrevem o desempenho de entrega de software por quatro indicadores, mas foram estabelecidas a partir de questionários autorrelatados por equipes corporativas, o que as torna não auditáveis e de aplicação incerta a projetos open-source. Calculamos as quatro métricas para 350 repositórios populares do GitHub em uma janela fixa de 12 meses, usando apenas dados públicos da API, com releases como proxy de deploy e falhas de CI como proxy de falha. Validamos os proxies contra uma amostra-ouro de 60 repositórios e 299 releases, rotuladas de forma independente por três avaliadores. A mediana da frequência de deploy é de 0,42 release por semana e a do tempo de recuperação é de 2,56 horas, mas a taxa de falha depende de como a falha é operacionalizada: o proxy de CI e o de entrega correlacionam-se a apenas* $\rho = 0{,}107$ *e discordam sobre a categoria DORA de 20,6% da amostra. Uma análise de sensibilidade sobre três definições de deploy move entre 25% e 33% dos repositórios de categoria. Concluímos que classificações DORA obtidas de dados públicos são contingentes a escolhas operacionais raramente explicitadas.*

1. Introdução

As métricas DORA, propostas pelo programa *DevOps Research and Assessment* [DORA 2024], descrevem o desempenho de entrega de uma equipe por quatro indicadores: frequência de *deploy*, *lead time* para mudanças, taxa de falha das mudanças

(CFR) e tempo de recuperação. Com os quatro, a equipe é classificada como *Elite*, *High*, *Medium* ou *Low*, e a literatura associa as categorias superiores a práticas como integração contínua e implantação em pequenos lotes [Forsgren et al. 2018]. As faixas vieram de questionários autorrelatados por equipes corporativas, que não podem ser auditados, e não se sabe o que significam em projetos *open-source*, cujo ritmo de publicação é decidido por mantenedores voluntários.

Neste trabalho calculamos as quatro métricas apenas com dados públicos do GitHub, para 350 repositórios populares que usam GitHub Actions, em uma janela fixa de 12 meses. Como a API não informa quando uma mudança chegou à produção nem quando um defeito afetou usuários, usamos dois *proxies*: a publicação de uma *release* aproxima o *deploy*, e a falha de um *workflow* de CI aproxima a falha de uma mudança. Também verificamos esses *proxies*: três avaliadores rotularam uma amostra-ouro de 60 repositórios e 299 *releases*, contra a qual avaliamos a heurística que identifica *releases* corretivas, e comparamos três definições de *deploy* para ver o quanto as conclusões mudam com elas.

1.1. Questões de pesquisa

RQ 01 Com que frequência são feitos *deploys* nesses projetos?
RQ 02 Qual o *lead time* para mudanças, nas variantes (a) e (b)?
RQ 03 Qual a taxa de falha das mudanças, nas variantes (a) e (b)?
RQ 04 Qual o tempo de recuperação após uma falha?
RQ 05 Frequência de *deploy* e taxa de falha se correlacionam?
RQ 06 Que características dos repositórios se associam ao desempenho?
RQ 07 A classificação muda se o *deploy* for definido de outra forma?
RQ 08 Qual a proporção de *releases* corretivas (*rework rate*)?

A Seção 2 descreve a metodologia, a Seção 3 os resultados, a Seção 4 a discussão, a Seção 5 as ameaças à validade [Wohlin et al. 2012] e a Seção 6 as conclusões.

1.2. Hipóteses

Antes da coleta, registramos hipóteses para as RQ 03 a 05; as demais questões foram tratadas como exploratórias.

RQ 03. Esperávamos o CFR (a) entre 10% e 25% na mediana: os *runs* vêm de *push* no *default branch*, já validados por um *pull request*, mas testes instáveis e dependências externas ainda devem falhar. Para o CFR (b), esperávamos valores parecidos ou menores, com muitos repositórios em 0%, e pouca concordância entre as variantes, porque a (a) mede falha de *pipeline* e a (b) aproxima falha percebida pelo usuário.

RQ 04. Esperávamos uma mediana de algumas horas, com a maioria na faixa *High*, uma cauda longa (de minutos a dias, já que mantenedores trabalham de forma voluntária e assíncrona) e uma proporção não trivial de episódios censurados.

RQ 05. Para o CFR (a), esperávamos correlação fraca negativa ou nula, como afirma o DORA ao dizer que velocidade e estabilidade não são um *trade-off*. Para o CFR (b), esperávamos correlação positiva, em parte por artefato da métrica: quem publica muitas *releases* tem mais chance de publicar um *patch* em até 7 dias após qualquer uma delas.

2. Metodologia

2.1. Fonte de dados, janela e amostra

Os dados vêm da API REST do GitHub, consultada por um *pipeline* próprio em Python (só com a biblioteca `requests`) que roda com um único comando. Cada resposta fica em um *cache* local indexado pelo *hash* da URL, então uma coleta interrompida continua de onde parou; a execução final fez 11.989 chamadas e reaproveitou 58.459 respostas do *cache*. O cliente espera o *reset* da cota e repete erros 5xx com *backoff* exponencial. A janela vai de 01/10/2025 a 30/09/2026, inclusiva, e só entram *releases* e *workflow runs* do *default branch* criados nela.

A busca devolve no máximo 1.000 repositórios por consulta, e só a faixa de 1.000 a 1.500 estrelas tem 20.493. Dividimos então a busca em sete faixas de estrelas, das quais entraram os 1.000 mais estrelados de cada (7.000 candidatos), avaliados alternando as faixas. A listagem de *runs* também para em 1.000, então os *runs* são pedidos mês a mês, e o mês que passa do teto é dividido ao meio até caber. Cada candidato parou no primeiro filtro em que falhou, e a coleta parou ao completar 350 aprovados (Tabela 1).

Tabela 1. Funil de seleção da amostra.

Etapa	Restantes	Descart.	Motivo do descarte
Candidatos da busca (7 faixas)	7.000	–	–
Avaliados	1.565	5.435	não avaliados (amostra completa)
Usam GitHub Actions	1.057	508	nenhum workflow
≥ 5 releases na janela	439	618	menos de 5 releases
Runs mensuráveis	433	6	> 1.000 *runs* em um dia
≥ 50 *runs* válidos	352	81	menos de 50 *runs* válidos
Coleta sem erro	350	2	erro definitivo da API

Os 5.435 não avaliados nunca foram consultados, porque a amostra já estava completa. Os seis repositórios com mais de 1.000 *runs* em um único dia saíram porque a API não devolve todos os *runs* desses dias, e o CFR (a) e a recuperação seriam calculados sobre dados truncados. A amostra final tem de 1.493 a 291.317 estrelas (mediana de 11.944); as linguagens mais frequentes são TypeScript (73), Python (72), Go (45), Rust (39) e JavaScript (28), e a mediana é de 22 *releases*, 688 *runs* válidos e 6,0 anos de idade. Ela não é aleatória: dentro de cada faixa só os mais estrelados viraram candidatos, e os mais populares passaram mais nos filtros. Os resultados valem para repositórios populares que usam GitHub Actions e publicam *releases*. O dicionário de dados, com tipo, unidade e origem de cada coluna, está no repositório.

2.2. Definições operacionais

Deploy é uma *release* publicada (`draft = false`); pré-*releases* e *tags* entram só como variantes na RQ 07. A frequência é o número de *releases* na janela dividido pelo número de semanas. Os *commits* de cada *release* R vêm de `compare/{anterior}...{R}`, com a data `commit.author.date`; o *lead time* (a) é a mediana, entre as *releases*, de "publicação de R menos o *commit* mais antigo de R", e o (b) é a mediana sobre todos os *commits* de todas as *releases*. *Releases* sem anterior ou com `compare` em 404 são ignoradas e contadas. O CFR (a) é a proporção de *runs* de `push` com falha (`failure`,

`timed_out`, `startup_failure`) entre falhas e sucessos. O tempo de recuperação é calculado dentro de cada *workflow*, da primeira falha após um sucesso até o próximo sucesso; episódios que não terminam na janela são censurados e reportados. A classificação usa os cortes fixos da disciplina, com nota de 1 a 4 por métrica, e a categoria geral é a mediana das notas, arredondada para baixo.

O CFR (b) marca uma *release* R como falha se ela for seguida, em até 7 dias, por uma *release* corretiva, e divide as falhas pelas *releases* avaliadas; as publicadas nos últimos 7 dias da janela ficam fora do denominador (censura). Para identificar *releases* corretivas testamos quatro heurísticas sobre a versão (SemVer) e o assunto dos *commits*: v1 (só *patch* ou algum *commit* `fix`/`revert`/`hotfix`), v2 (só *patch* ou ao menos metade dos *commits* são correções), v3 (*patch* com algum *commit* de correção, ou ao menos metade de correções) e v4, descrita abaixo. Comparamos cada uma com o consenso da amostra-ouro (Seção 2.3) por precisão, recall e F1 da classe "corretiva", com limiar F1 $\geq$ 0,70 (Tabela 2).

Tabela 2. Heurísticas contra o consenso (299 *releases*, 111 corretivas).

Versão	VP	FP	FN	VN	Precisão	Recall	F1
v1	110	170	1	18	0,39	0,99	0,56
v2	84	74	27	114	0,53	0,76	0,62
v3	71	67	40	121	0,51	0,64	0,57
v4	81	57	30	131	0,59	0,73	**0,65**

As três primeiras partem da mesma suposição, de que toda mudança de *patch* é correção; a v1 acerta quase todas as corretivas (recall 0,99) dizendo "sim" para quase tudo (precisão 0,39). A v4 troca esse critério por dois sinais: as *release notes* só têm seção de correção (nesse caso, 88% das *releases* da amostra-ouro são corretivas, contra 56% quando o único sinal é o *patch*), ou a versão é *patch* e há ao menos um *commit* de correção fora de manutenção, dependências e infraestrutura (`chore`, `ci`, `docs`, `test`, `bump`). Ela foi a escolhida, com F1 de 0,65: os falsos positivos caem de 170 para 57. Nenhuma versão atingiu 0,70, o que tratamos como limitação na Seção 5. A v4 não usa limiar numérico ajustado na amostra-ouro, e por isso pode ser aplicada a outra amostra sem calibração.

2.3. Validação manual

Sorteamos 60 repositórios da amostra (`df.sample(n=60, random_state=42)`, após ordená-los pelo nome) e, de cada um, 5 *releases* com *release* anterior e `compare` disponível; um repositório tinha só 4, e a amostra ficou com 299. Os três autores rotularam de forma independente, sem consultar uns aos outros nem a saída da heurística: (i) o tipo do projeto (biblioteca, aplicação, CLI ou outro); (ii) se as *releases* representam entregas reais ao usuário; e (iii) se cada *release* é corretiva, lendo as *release notes* e os *commits*. Medimos a concordância pelo kappa de Fleiss, interpretado pela escala de Landis e Koch [Landis and Koch 1977] (Tabela 3). O consenso é a maioria simples; empates 1-1-1 iriam para reunião, mas não houve nenhum.

O menor kappa, o de entregas reais, vem do desbalanceamento: o consenso pôs 59 dos 60 repositórios em "sim", e com isso uma única divergência derruba o coeficiente, embora a concordância bruta seja de 98,3%. O tipo do projeto teve mais divergências, em

Tabela 3. Concordância entre os três avaliadores na amostra-ouro.

Dimensão	Itens	Divergências	κ	Concordância
Tipo do projeto	60	9	0,835	Quase perfeita
Releases são entregas reais	60	1	0,745	Substancial
Release corretiva	299	26	0,876	Quase perfeita

projetos que entregam ao mesmo tempo um servidor e um SDK e no alcance da categoria "outro". Em *release* corretiva, as 26 divergências vêm de *releases* que misturam correções e novidades. Como a heurística é avaliada contra esse consenso, o kappa de 0,876 serve de teto prático para ela.

3. Resultados

Como as distribuições são assimétricas e têm cauda longa, reportamos mediana e IQR. Para saber o quanto cada número é estável, calculamos também intervalos de confiança de 95% por *bootstrap* percentil (2.000 reamostragens dos repositórios, semente fixa) para as medianas, as correlações e as proporções de mudança de categoria (Tabela 4).

Tabela 4. Estatística descritiva das métricas DORA (350 repositórios) e IC 95% da mediana.

Métrica	n	Q1	Mediana	Q3	IQR	IC 95% da mediana
Frequência (releases/semana)	350	0,19	0,42	1,04	0,84	[0,35; 0,46]
Lead time (a) (h)	350	50,8	251,7	848,4	797,6	[196,7; 337,3]
Lead time (b) (h)	350	65,4	284,0	845,4	780,1	[206,8; 331,6]
CFR (a), proxy de CI	350	0,021	0,049	0,121	0,100	[0,041; 0,065]
CFR (b), proxy de entrega	350	0,052	0,200	0,416	0,364	[0,174; 0,227]
Recuperação (h)	330	0,98	2,56	9,53	8,55	[2,08; 3,30]

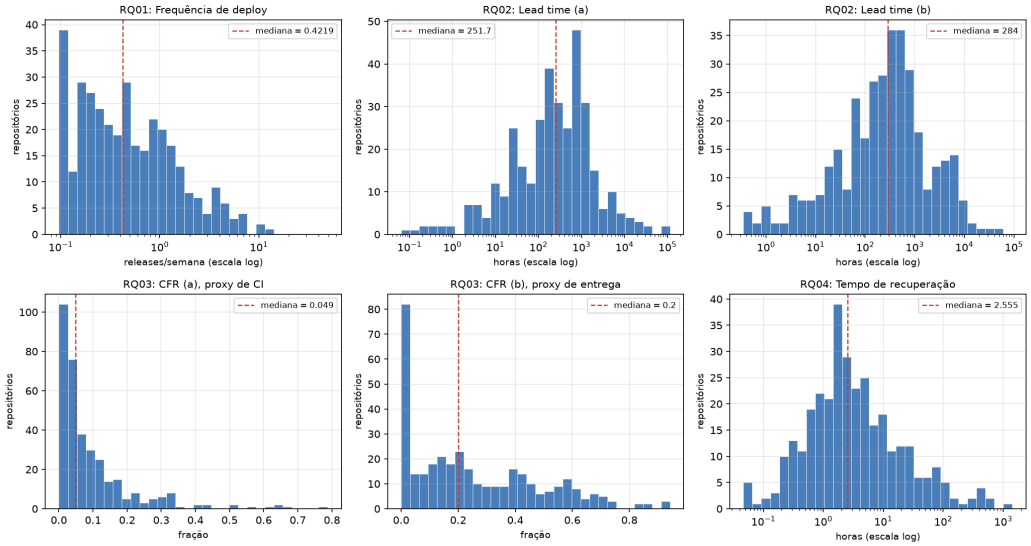


Figura 1. Distribuição das seis métricas. Frequência, *lead time* e recuperação em escala logarítmica; a linha tracejada marca a mediana.

RQ 01. A mediana é de 0,42 *release* por semana, cerca de uma a cada duas semanas e meia. 260 repositórios (74,3%) publicam menos de uma por semana, e só

5 (1,4%) alcançam o patamar *Elite* de sete ou mais; 107 (30,6%) ficam em *Low* e 153 (43,7%) em *Medium*.

RQ 02. As variantes ficaram próximas: 251,7 h (10,5 dias) na (a) e 284,0 h (11,8 dias) na (b), com ICs quase sobrepostos e correlação de Spearman de 0,744 entre elas. Elas divergem em repositórios específicos por construção: a variante (a) usa o *commit* mais antigo de cada *release*, então um único *commit* antigo "esquecido" num ramo faz o *lead time* daquela *release* explodir, enquanto na (b) ele é só um valor entre muitos e quase não move a mediana. Nas medianas da amostra as duas ficaram próximas, mas o efeito aparece na cauda: o máximo é de $1{,}1 \times 10^5$ h na variante (a) e de $9{,}3 \times 10^4$ h na (b). Pelos cortes DORA, 57 repositórios ficam em *Elite*, 89 em *High*, 103 em *Medium* e 101 em *Low*.

RQ 03. A mediana do CFR (a) é 0,049 (IQR 0,100), e 13 repositórios (3,7%) não têm nenhum *run* com falha; pelos cortes, 283 (80,9%) ficam em *Elite*. A mediana do CFR (b) é 0,200 (IQR 0,364), quatro vezes maior, com 80 repositórios (22,9%) em zero. As duas variantes ordenam os repositórios de forma muito diferente: a correlação de Spearman entre elas é de 0,107 ($p = 0{,}045$), e o IC 95% vai de praticamente zero ($-0{,}0001$) a 0,21.

RQ 04. A mediana é de 2,56 h (IQR 8,55 h) nos 330 repositórios com ao menos um episódio de falha seguida de sucesso. 196 (59,4%) ficam em *High* (de 1 h a menos de um dia), 86 (26,1%) em *Elite* e 48 (14,5%) em 24 h ou mais; o máximo é de 1.418 h. Os 20 repositórios (5,7%) sem nenhum episódio ficam fora da mediana; a maioria deles não teve falha alguma, como o `arendst/Tasmota`, com 569 *runs* válidos. A censura foi desprezível: mediana 0, terceiro quartil 0,0033, e 70,9% dos repositórios sem nenhum episódio censurado.

Classificação geral. 25 repositórios (7,1%) são *Elite*, 143 (40,9%) *High*, 159 (45,4%) *Medium* e 23 (6,6%) *Low*. A nota de CFR puxa a categoria para cima (*Elite* em 80,9% dos casos) e a de frequência para baixo (*Low* ou *Medium* em 74,3%).

RQ 05. A correlação de Spearman entre a frequência de *deploy* e o CFR é fraca na variante (a), $\rho = 0{,}127$ ($p = 0{,}017$; IC 95% de 0,03 a 0,22), e forte na (b), $\rho = 0{,}613$ ($p < 0{,}001$; IC 95% de 0,53 a 0,68), com $n = 350$ nas duas (Figura 2). O ρ da variante (a) explica menos de 2% da variância dos postos.

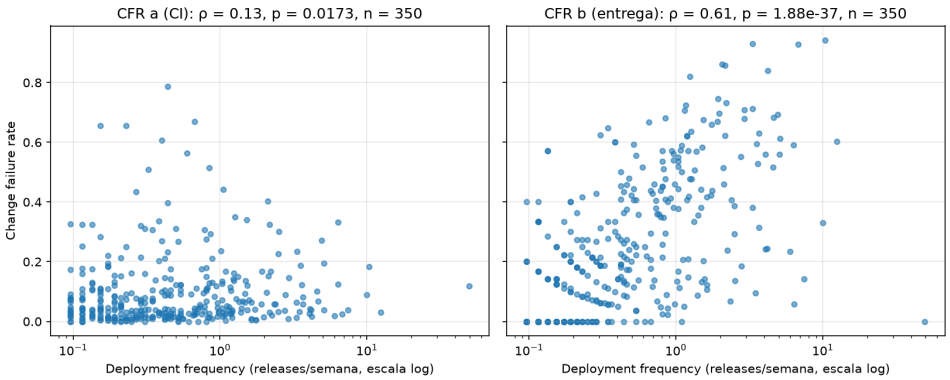


Figura 2. Frequência de *deploy* (escala log) contra o CFR, nas duas variantes.

RQ 06. Comparamos cada métrica entre os grupos de cinco fatores: linguagem (as cinco mais frequentes e "Outras"), quartis de estrelas, de contribuidores e de idade, e o tipo do projeto rotulado na amostra-ouro. Todos os fatores têm três ou mais grupos, então usamos Kruskal-Wallis com ε^2 como tamanho de efeito, e corrigimos os 30 p-valores juntos pelo método de Holm [Holm 1979]. Doze testes permanecem significativos (Tabela 5). A idade aparece em cinco métricas, a linguagem em quatro, os contribuidores em duas e as estrelas em uma; nenhum teste sobre o tipo do projeto foi significativo, mas ele só existe para os 60 repositórios da amostra-ouro. O maior efeito, $\varepsilon^2 = 0{,}288$, é o da idade sobre o *lead time* (a).

Tabela 5. Testes significativos após a correção de Holm, por tamanho de efeito.

Fator	Métrica	p (Holm)	ε^2	Fator	Métrica	p (Holm)	ε^2
Idade	Lead time (a)	$<$ 0,001	0,288	Linguagem	Lead time (a)	$<$ 0,001	0,099
Idade	Lead time (b)	$<$ 0,001	0,251	Linguagem	Frequência	$<$ 0,001	0,097
Contribuidores	Lead time (b)	$<$ 0,001	0,171	Linguagem	CFR (b)	$<$ 0,001	0,084
Idade	CFR (b)	$<$ 0,001	0,164	Linguagem	Lead time (b)	0,001	0,078
Idade	Frequência	$<$ 0,001	0,106	Idade	Recuperação	0,016	0,051
Contribuidores	Lead time (a)	$<$ 0,001	0,104	Estrelas	Lead time (b)	0,014	0,049

RQ 07. Recalculamos a classificação sob três definições de *deploy*: C1 (referência), com *releases*, *lead time* (a) e CFR (a); C2, com *releases* e pré-*releases* e as variantes (b); e C3, com *tags*, *lead time* (b) e CFR (a). Entre um quarto e um terço dos repositórios mudam de categoria entre as combinações, e o kappa ponderado fica entre 0,52 e 0,66 (Tabela 6). A faixa *Elite* é a mais sensível: vai de 8 repositórios em C2 a 33 em C3.

Tabela 6. Classificação DORA sob as três definições de *deploy* e concordância entre elas (IC 95% por *bootstrap*).

Categoria	C1	C2	C3	Par	Mudam de categoria	Kappa pond.
Elite	25	8	33	C1 $\times$ C2	32,9% [28,0; 38,0]	0,515
High	143	134	164	C1 $\times$ C3	25,4% [21,1; 30,0]	0,657
Medium	159	186	138	C2 $\times$ C3	28,0% [23,4; 32,6]	0,602
Low	23	22	15			

RQ 08. O *rework rate*, proporção de *releases* corretivas sobre o total de *releases* da janela, tem mediana de 0,500 (Q1 0,225; Q3 0,726), 2,5 vezes a do CFR (b). As duas medidas se correlacionam ($\rho = 0{,}522$, $p < 0{,}001$), mas 309 dos 350 repositórios têm *rework rate* maior que o CFR (b), e 21 não publicaram nenhuma *release* corretiva. A diferença vem da definição: o *rework rate* conta toda *release* corretiva, e o CFR (b) só as que vieram em até 7 dias depois de outra.

4. Discussão

As hipóteses sobre a taxa de falha erraram nas duas direções. Esperávamos o CFR (a) entre 10% e 25%, e ele ficou em 4,9%. Os *runs* contados são de *push* no *default branch*, ou seja, de mudanças que já passaram por um *pull request* com CI; a métrica mede a falha do que já foi aprovado, que em projetos populares com revisão obrigatória é baixa. Para o CFR (b) esperávamos valores iguais ou menores, e a mediana foi quatro vezes maior. Parte disso vem do instrumento: a heurística adotada tem mais falsos positivos (57) que

falsos negativos (30) na amostra-ouro, então o valor de 0,200 tende a superestimar o real. A única hipótese confirmada na RQ 03 foi a de que as variantes concordariam pouco.

As duas variantes do CFR medem coisas diferentes. Com $\rho = 0{,}107$ e IC que encosta no zero, um projeto bem avaliado por uma variante pode ser mal avaliado pela outra. Recalculando a classificação geral com o CFR (b) no lugar do (a), 72 dos 350 repositórios (20,6%) mudam de categoria e a faixa *Elite* cai de 25 para 8. No *lead time* foi diferente ($\rho = 0{,}744$ e medianas próximas), porque as duas variantes medem o mesmo evento de formas diferentes, enquanto as do CFR medem eventos distintos: falha de *pipeline* antes da entrega e correção depois dela.

A escala DORA e as releases open-source. A frequência é a dimensão em que a amostra vai pior, com 74,3% dos repositórios abaixo de uma *release* por semana. Isso diz mais sobre o *proxy* do que sobre a capacidade de entrega: uma *release* é um marco de versionamento, muitas vezes mensal ou trimestral, enquanto o *deploy* do DORA [DORA 2024] é a chegada de uma mudança à produção, e o patamar *Elite* de sete por semana pressupõe entrega contínua em ambiente corporativo [Forsgren et al. 2018]. Como a categoria geral é a mediana de quatro notas, ela acaba definida pelo par CFR (a) e recuperação, as dimensões em que a amostra vai melhor.

Velocidade e estabilidade (RQ 05). As duas previsões se confirmaram na direção. Na variante (a), a correlação é positiva mas fraca, e o IC exclui o zero por pouco: publicar com mais frequência quase não vem acompanhado de mais falhas de CI, o que é compatível com a tese do DORA de que velocidade e estabilidade não são um *trade-off*. Na variante (b), a correlação forte é em boa parte artefato da métrica, porque quem publica toda semana tem muito mais chances de ter uma *release* corretiva em até 7 dias do que quem publica a cada trimestre. Com apenas a variante (b), a conclusão seria o oposto do que o DORA sustenta.

O que explica as diferenças (RQ 06). A idade é o fator mais associado ao desempenho: projetos mais antigos acumulam mais tempo entre o *commit* mais antigo e a publicação (ε^2 de 0,288 e 0,251 no *lead time*) e publicam menos. A popularidade quase não explica nada: o número de estrelas só aparece no *lead time* (b), com o menor efeito ($\varepsilon^2 = 0{,}049$).

As conclusões são robustas? (RQ 07 e RQ 08). Depende da RQ. A do *lead time* (RQ 02) se mantém nas duas variantes. As da taxa de falha (RQ 03 e 05) e a classificação geral não se mantêm: trocar a variante do CFR muda a categoria de 20,6% dos repositórios, e trocar a unidade de *deploy* muda entre 25% e 33%, e nenhum desses ICs desce abaixo de 21%. O *rework rate* reforça a leitura do CFR (b): metade das *releases* corrige algo, mas só uma em cinco corrige algo publicado na semana anterior, então a maior parte da correção é manutenção de rotina.

5. Ameaças à Validade

Seguimos as quatro categorias de Wohlin et al. [Wohlin et al. 2012].

Construto. *Release* não é *deploy*, e falha de CI não é falha em produção: um *run* que falha costuma ser um erro barrado antes de chegar ao usuário. A validação manual sustentou o primeiro *proxy* (as *releases* representam entregas reais em 59 dos 60 repositórios da amostra-ouro), mas a heurística de *release* corretiva não atingiu o F1 de

0,70 (0,65 na v4), e esse erro passa para o CFR (b), a RQ 08 e a C2 da RQ 07. Os próprios avaliadores divergiram em 26 de 299 *releases*. Rebase e *squash merge* reescrevem o `commit.author.date` e afetam o *lead time* em direção desconhecida.

Interna. As análises são correlacionais. A idade se correlaciona com a frequência ($\rho = -0{,}306$), assim como as estrelas ($\rho = 0{,}162$) e os contribuidores ($\rho = 0{,}119$), e esses fatores não são controlados nas RQ 01 a 04. O filtro de inclusão (5 *releases* e 50 *runs*) deixa só projetos que já versionam e automatizam com regularidade, e o descarte dos seis repositórios com mais de 1.000 *runs* por dia tira da amostra os de maior atividade de CI.

Externa. A amostra não é aleatória: entraram os 1.000 mais estrelados de cada faixa, e os menos estrelados de cada faixa nunca foram observados. Os resultados valem para repositórios populares com GitHub Actions, e não se transferem para equipes corporativas, que são o contexto das faixas DORA. A janela é única, de 12 meses, e não separa característica estável de efeito do período.

Conclusão. Usamos mediana, IQR e testes não paramétricos por causa da assimetria, e a correção de Holm nos 30 testes da RQ 06. O tipo do projeto só existe para 60 repositórios, e os testes sobre ele têm pouco poder. Os ICs por *bootstrap* mostram que as medianas e as proporções de mudança de categoria são estáveis, mas que a correlação entre as variantes do CFR não se distingue com segurança de zero.

6. Conclusão

Medimos as quatro métricas DORA em 350 repositórios *open-source* populares usando somente dados públicos do GitHub. A amostra publica pouco (mediana de 0,42 *release* por semana), leva cerca de dez dias entre o *commit* mais antigo e a publicação, falha pouco no CI (mediana de 0,049) e se recupera rápido (mediana de 2,56 h); 86,3% dos repositórios são *High* ou *Medium*.

Esses valores dependem bastante de decisões de medição. As duas variantes da taxa de falha correlacionam-se a $\rho = 0{,}107$ e discordam sobre a categoria de 20,6% da amostra; trocar a definição de *deploy* move entre 25% e 33% dos repositórios, e a faixa *Elite* varia de 8 a 33 projetos. A correlação entre frequência e taxa de falha é fraca no CI ($\rho = 0{,}127$) e forte na entrega ($\rho = 0{,}613$), em boa parte por artefato da métrica. Na validação manual, o *proxy* de *release* se sustentou (59 de 60 repositórios), mas a melhor heurística de *release* corretiva ficou em F1 de 0,65, numa dimensão em que os próprios avaliadores divergiram em 26 de 299 casos. Uma classificação DORA de projetos *open-source* só pode ser interpretada junto com as escolhas operacionais que a produziram.

Como trabalhos futuros, sugerimos usar projetos com ambiente de produção observável (por exemplo, imagens de contêiner com data de implantação) no lugar do *proxy* de *release*, estender a janela para além de 12 meses e ampliar a amostra-ouro para treinar e validar a heurística em conjuntos disjuntos.

Disponibilidade dos dados

Código, dados e roteiros de análise: `https://github.com/candian15/LabExpGRP` (pasta `LAB03`). Quadro do GitHub Projects: `https://github.com/users/candian15/projects/2`.

Referências

DORA (2024). DORA's software delivery metrics: the four keys. https://dora.dev/guides/dora-metrics-four-keys/. Acesso em 2026.

Forsgren, N., Humble, J., and Kim, G. (2018). *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press, Portland, OR.

Holm, S. (1979). A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70.

Landis, J. R. and Koch, G. G. (1977). The measurement of observer agreement for categorical data. *Biometrics*, 33(1):159–174.

Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., and Wesslén, A. (2012). *Experimentation in Software Engineering*. Springer, Berlin, Heidelberg.